

Financial Document Intelligence & RAG on Azure

An end-to-end delivery plan: ingestion through evaluation, with the RAGAS metric suite promoted from an offline notebook into a first-class product surface.

CORPUS	SEGMENTS	EFFORT	BACKEND	
100–300 SEC filings	9 phases	6–7 weeks p/t	FastAPI · Python 3.12	
FRONTEND	EVAL			
Next.js 15 · TS	RAGAS + Azure AI Eval			

CONTENTS		
Scope & the gap	System architecture	Tech stack
Repo & services	Backend design	Frontend design
Evaluation engine	RAGAS metric suite	Evaluation UI
Security & ops	Phased build	Cost & risk

What exists, and what this plan adds

The existing build guide is a strong *pipeline* design — parsing, chunking, indexing, hybrid retrieval, citations. It is not yet an application plan. Three things are missing, and they are the three that turn a notebook into a portfolio piece.

There is no application layer. The guide ends at "answer + inline citations" with no API contract, no session model, no streaming design, and no document viewer to render a citation back to a page and bounding box.

Evaluation is specified as a script. RAGAS is named, metrics are listed, and the ablation table is described as a README artifact. But an eval run that lives in a CSV is invisible — you can't compare two configs at a glance, can't see which question type is dragging the mean down, and can't show it to anyone. This plan makes evaluation a product surface with its own data model, service, and five screens.

Nothing is measured in production. Offline eval on a 60-question golden set tells you nothing about the queries real users actually type. The plan adds online sampled evaluation writing into the same metric store as the offline runs.

SCOPE CHANGE WORTH NAMING

The guide budgets 3–4 weeks. Adding a real frontend, an evaluation service, and the eval dashboard puts this at **6–7 weeks part-time**. If time is fixed at four weeks, cut the finance add-ons (Phase 9) and the online eval loop, not the eval dashboard — the dashboard is the differentiating artifact.

Design principles this plan holds to

- **Every config is named and versioned.** Retrieval is parameterised by a `RetrievalConfig` record (chunker, k, fusion, ranker, model deployments). Every answer and every eval result carries its config id. Ablations become a database query, not a manual bookkeeping exercise.
- **The trace is the unit of work.** One query produces one trace: query, filters, per-retriever candidates with scores, fused set, prompt, response, citations, validation outcome, token counts, stage latencies. Offline eval, online eval, the debug UI, and App Insights all read the same object.
- **Metrics are rows, not columns.** Storing `(run_id, question_id, metric_name, value, reason)` long-form means adding a RAGAS metric needs no migration and no UI change.
- **The judge is never the generator.** A stronger, separately-deployed model scores answers, to avoid self-preference bias inflating faithfulness.

02 • ARCHITECTURE

The two planes and the spine between them

Ingestion is a queue-driven batch job; query is a streaming request path. They share exactly one thing — the search index — and are deployed, scaled, and failed independently.

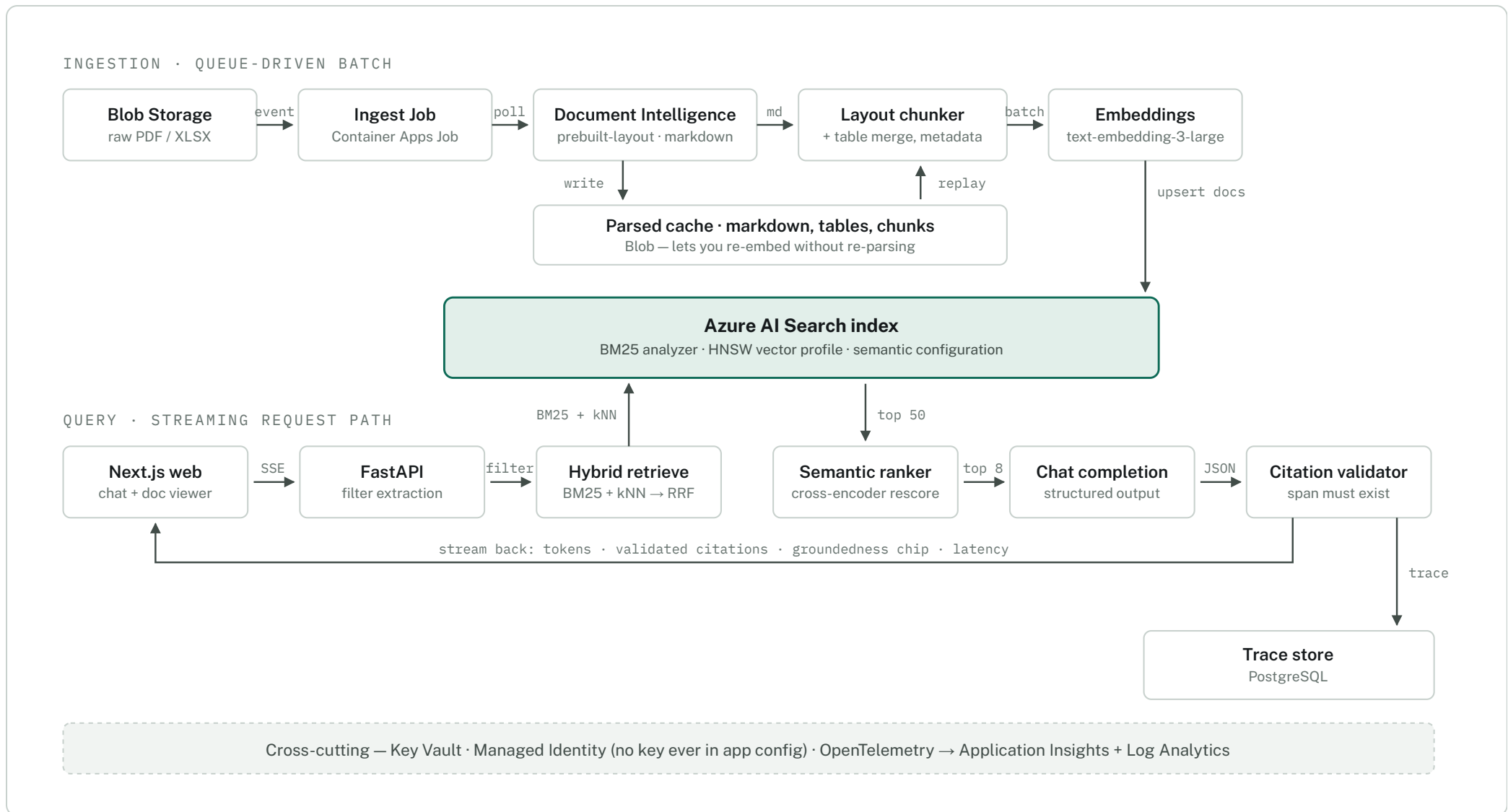


Figure 1. The parsed cache is the difference between a two-hour re-index and a ten-minute one: parsing is the slow, billed, non-deterministic step, so its output is persisted before chunking. Note that retrieval makes *one* call to the index carrying both a text query and a vector query — Azure AI Search performs the RRF fusion server-side; you do not fuse two responses in application code.

Technology choices, and what each one is doing for you

Two languages, one cloud. Python owns everything touching documents, retrieval and evaluation; TypeScript owns the browser. No shared runtime, no ORM in the frontend, one typed contract between them generated from OpenAPI.

BACKEND RUNTIME LangPython 3.12 APIFastAPI + Uvicorn Native async, SSE streaming, and OpenAPI generated from the same Pydantic models the pipeline uses. ModelsPydantic v2 One schema set for chunks, citations, traces, eval records — validated at every boundary. Depsuv + lockfile JobsContainer Apps Jobs Ingestion and eval runs are jobs, not endpoints. Scale-to-zero, queue-triggered, resumable.	AI & RETRIEVAL ParseAI Document Intelligence <pre>prebuilt-layout</pre>, markdown output mode. Embedtext-embedding-3-large Shortened to 1024 dims — measure the recall cost, then keep or revert. IndexAzure AI Search BM25 + HNSW + semantic ranker in one service. GenerateAzure OpenAI (chat) Structured outputs / JSON schema, not prose parsing. JudgeSeparate deployment Stronger model, its own quota, so eval never competes with serving traffic.
DATA RelationalPostgreSQL Flexible Server Documents, golden set, eval runs, metric rows, traces, extractions. The eval UI is fundamentally a set of aggregate queries. MigrationsAlembic ObjectsBlob Storage Raw docs, parsed cache, page renders for the citation viewer. CacheAzure Cache for Redis Embedding cache and — the big one — judge-verdict cache.	FRONTEND FrameworkNext.js 15, App Router Route handlers proxy the API so no token reaches the browser; RSC for dashboard shells. LangTypeScript strict StylingTailwind v4 + shadcn/ui Server stateTanStack Query TablesTanStack Table Ablation matrix and per-question drilldown need sorting, grouping, column pinning. ChartsRecharts PDFreact-pdf (pdf.js) Renders the cited page with a bounding-box overlay.
EVALUATION CoreRAGAS Metric suite + synthetic testset generation to bootstrap the golden set. Azureazure-ai-evaluation Groundedness Pro, retrieval, safety evaluators; results push to Foundry. CustomIn-repo evaluators Citation validity, numeric exactness, refusal accuracy. HarnessTyper CLI + job Same code path runs locally, in CI, and as a scheduled job.	PLATFORM & DELIVERY ComputeAzure Container Apps API, web, and jobs in one environment. IaCBicep modules Per-environment parameter files; the index schema is deployed as code too. IdentityEntra ID + Managed Identity SecretsKey Vault + RBAC data-plane roles CI/CDAzure DevOps Lint → test → build → deploy, with an eval quality gate before promotion. TelemetryOpenTelemetry → App Insights

Table 1. Deliberate omissions: no LangChain or LlamaIndex in the request path — the pipeline is six explicit steps and a framework would only obscure the retrieval call you most want to control. RAGAS pulls LangChain in as a dependency for its judge wrappers; that is confined to the eval service.

Choices worth defending out loud

DECISION	CHOSEN	REJECTED ALTERNATIVE	REASONING
Vector store	Azure AI Search	pgvector, Qdrant	You need BM25, vectors, RRF and a hosted cross-encoder in one hop. pgvector gives you vectors and then you are hand-building fusion and reranking.
Metadata DB	PostgreSQL	Cosmos DB	The eval surface is aggregation-heavy — group by config, question type, metric. That is SQL's home turf, and it keeps eval cost near zero.
Streaming	Typed SSE events	Plain token stream	The client needs more than tokens: citation objects, validation verdicts, and stage timings arrive as distinct event types the UI renders differently.
Ingestion	Queue + job	Search indexer + skillset	The built-in indexer is faster to stand up but you lose control of chunking, and cross-page table merging is not expressible in a skillset.
Frontend host	Container Apps	Static Web Apps	One environment, one identity model, one deployment pipeline for web and API.

Repository and service boundaries

One repo, four deployables, one shared Python library. The library is the important part: the chunker, the retrieval client, and the citation validator are imported identically by the API, the ingest job, and the eval runner — so an eval measures the code that actually serves traffic, not a reimplementaion of it.

azure-rag/ â"œâ"€â"€ apps/ â", â"œâ"€â"€ api/ â", â"œâ"€â"€ web/ â", â"œâ"€â"€ ingest/ â", â""â"€â"€ evaluator/ â"œâ"€â"€ libs/rag_core/ â", â"œâ"€â"€ parsing/ â", â"œâ"€â"€ chunking/ â", â"œâ"€â"€ indexing/ â", â"œâ"€â"€ retrieval/ â", â"œâ"€â"€ generation/ â", â"œâ"€â"€ citations/ â", â"œâ"€â"€ evaluation/ â", â""â"€â"€ telemetry/ â"œâ"€â"€ evals/ â", â"œâ"€â"€ golden_set.yaml â", â"œâ"€â"€ configs/ â", â""â"€â"€ thresholds.yaml â"œâ"€â"€ infra/ â""â"€â"€ .azure-pipelines/	FastAPI â€" chat, search, documents, eval read API Next.js â€" chat UI + evaluation dashboard Container Apps Job â€" queue consumer, resumable Container Apps Job + Typer CLI â€" offline & online eval DI client, table reconciliation, xlsx adapter layout chunker, context-header builder index schema as code, embedder, upserter hybrid query builder, filter extraction, RetrievalConfig prompt templates, structured output schema validator â€" the programmatic hallucination check RAGAS wiring, custom metrics, run orchestration OTel spans, trace model, cost accounting 60â€"80 curated Q/A with gold chunk ids named RetrievalConfig variants for ablation CI gate limits Bicep modules + per-env parameters ci.yml, cd.yml, eval-gate.yml, nightly-eval.yml
---	---

API surface and data contracts

Three groups of endpoints: the chat path, the corpus path, and the evaluation path. The evaluation endpoints are read-heavy and exist purely to feed the dashboard.

CHAT & RETRIEVAL

POST	/api/chat	SSE. Events: token , citations , validation , trace , done . Accepts an optional config_id so the UI can A/B a retrieval variant live.
POST	/api/search	Retrieval only, no generation. Returns each candidate with its BM25 rank, vector rank, RRF score and semantic rerank score. This powers the debug panel and is how you diagnose a bad answer.
GET	/api/traces/{trace_id}	Full trace for one answer — prompt, context set, timings, tokens, cost, validation outcome.

CORPUS

POST	/api/ingest	Enqueue documents. Returns a job id; ingestion itself is a job, not a request.
GET	/api/ingest/jobs/{id}	Per-document status: queued, parsing, chunked, embedded, indexed, failed — with the resume point.
GET	/api/documents	Filter by company, ticker, doc_type, fiscal_year. Backs the corpus browser and the filter chips.
GET	/api/documents/{id}/pages/{n}	Rendered page image plus the bounding regions for a given chunk — the citation viewer highlights the source span on the page.

EVALUATION

GET	/api/eval/golden-set	List and edit questions; each carries type, gold answer, gold chunk ids, and an <code>answerable</code> flag.
POST	/api/eval/runs	Start a run: config id(s), question subset, metric set. Fans out to the evaluator job.
GET	/api/eval/runs	Run history with aggregate scores — the run-history chart and the regression alert both read this.
GET	/api/eval/runs/{id}/summary	Aggregates pivoted by metric, and by metric × question type. The segment breakdown is the whole point.
GET	/api/eval/runs/{id}/questions	Per-question rows with every metric value, the judge's reason string, and the retrieved context.
GET	/api/eval/compare	N configs × M metrics, with cost and p95 latency per config. Renders the ablation matrix directly.
GET	/api/eval/live	Rolling online metrics from sampled production traffic over a time window.

Core data model

Nine tables. The two that carry the design are `retrieval_config` and `eval_metric_value`.

TABLE	KEY COLUMNS	WHY IT EXISTS
document	id, company, ticker, doc_type, fiscal_year, fiscal_quarter, source_url, page_count, parse_status	Corpus registry and ingestion state machine.
chunk	id, doc_id, section_path, page_start, page_end, contains_table, token_count, content_hash	Mirror of what is in the index, so you can join, audit and diff without querying Search.
retrieval_config	id, name, chunk_size, overlap, top_k, use_bm25, use_vector, use_semantic_ranker, embed_model, embed_dims, gen_model, prompt_version	The ablation axis. Every trace and every eval result references one. Comparing configs is a group-by.
trace	id, config_id, query, filters, retrieved (jsonb), answer, citations (jsonb), stage latencies, tokens, cost, refused	One row per answered query, online or offline. The single source for debugging and online eval.
golden_question	id, question, gold_answer, gold_chunk_ids, q_type, answerable, difficulty	The curated set. <code>q_type</code> drives the segment breakdown.
eval_run	id, config_id, question_set, metric_set, judge_model, started_at, status, trigger	One execution. <code>trigger</code> distinguishes manual, nightly, CI-gate and online.
eval_result	id, run_id, question_id, trace_id	Joins a question to the answer that was produced for it in that run.
eval_metric_value	result_id, metric_name, value, reason, judge_tokens	Long-form. New metric = new rows, never a migration. <code>reason</code> stores the judge's justification so the UI can show why a score is low.
extraction	doc_id, field, value, unit, source_chunk_id, identity_check_passed	Structured financials with accounting-identity validation.

THE CITATION VALIDATOR, CONCRETELY

The model returns `{answer, citations: [{chunk_id, quoted_span}]}`. For each citation: assert `chunk_id` was in the context set actually sent; normalise whitespace, quotes and digit separators on both sides; assert `quoted_span` occurs in that chunk's text. Failures are dropped from the rendered answer, counted into `citation_validation_failures`, and emitted as a `validation` SSE event so the UI can badge the answer honestly. This runs in single-digit milliseconds and needs no model call — it is the cheapest hallucination check available and it becomes one of your headline metrics.

06 • FRONTEND

Two applications behind one shell

The web app has two audiences that happen to be the same person: someone asking questions of the corpus, and someone tuning the system that answers them. They get separate route groups and share only the design system and the trace viewer.

ROUTE	SCREEN	WHAT IT DOES
/	Ask	Streaming chat. Answer text with inline superscript citation markers; hovering one previews the quoted span, clicking opens the source page. Filter chips (company, fiscal year, doc type) above the composer, pre-filled from what the query parser extracted so the user can see and correct the filter.
/	Answer footer	Per-answer quality strip: groundedness chip, citations validated <i>n/n</i> , chunks retrieved, latency, tokens, cost. This is evaluation in the product , not just in the dashboard.
/sources/[docId]	Document viewer	pdf.js render with the cited region boxed on the page; section outline from <code>section_path</code> ; jump-to-page from any citation.
/corpus	Corpus browser	Faceted list of ingested documents, parse status, chunk counts, and re-ingest action.
/debug/[traceId]	Retrieval inspector	Side-by-side BM25 / vector / RRF / semantic-rank orderings for one query, showing exactly where a chunk gained or lost position. The fastest way to answer "why did it say that".
/eval	Quality overview	Latest run KPIs vs previous, run-history trend, gate status. Detailed in the next section.
/eval/runs/[id]	Run detail	Metric table, score distributions, breakdown by question type.
/eval/compare	Ablation matrix	Configs × metrics with cost and latency columns.
/eval/questions/[id]	Question drilldown	Gold vs generated answer, retrieved context with gold-chunk marking, per-metric scores with judge reasoning.
/eval/live	Production quality	Rolling metrics on sampled real traffic.

Streaming contract

The client renders four event types differently, which is why a bare token stream is insufficient:

- `token` — appended to the answer body.
- `citations` — arrives once the structured output completes; markers become interactive at this point.

- `validation` — per-citation pass/fail; failed markers render struck through with a tooltip rather than silently disappearing.
- `trace` — timings, tokens, cost; populates the answer footer and links to the inspector.

Refusals are a first-class response, not an error: when the context does not support an answer the model returns an empty citation list and a refusal flag, and the UI shows a distinct "not found in corpus" state with the filters that were applied — which is usually the actual problem.

07 • EVALUATION ENGINE

Three producers, one metric store

Offline runs, online sampling, and the CI gate are the same evaluator invoked three ways. They write identical rows, which is what makes "did last night's change help?" and "is production still healthy?" the same query.

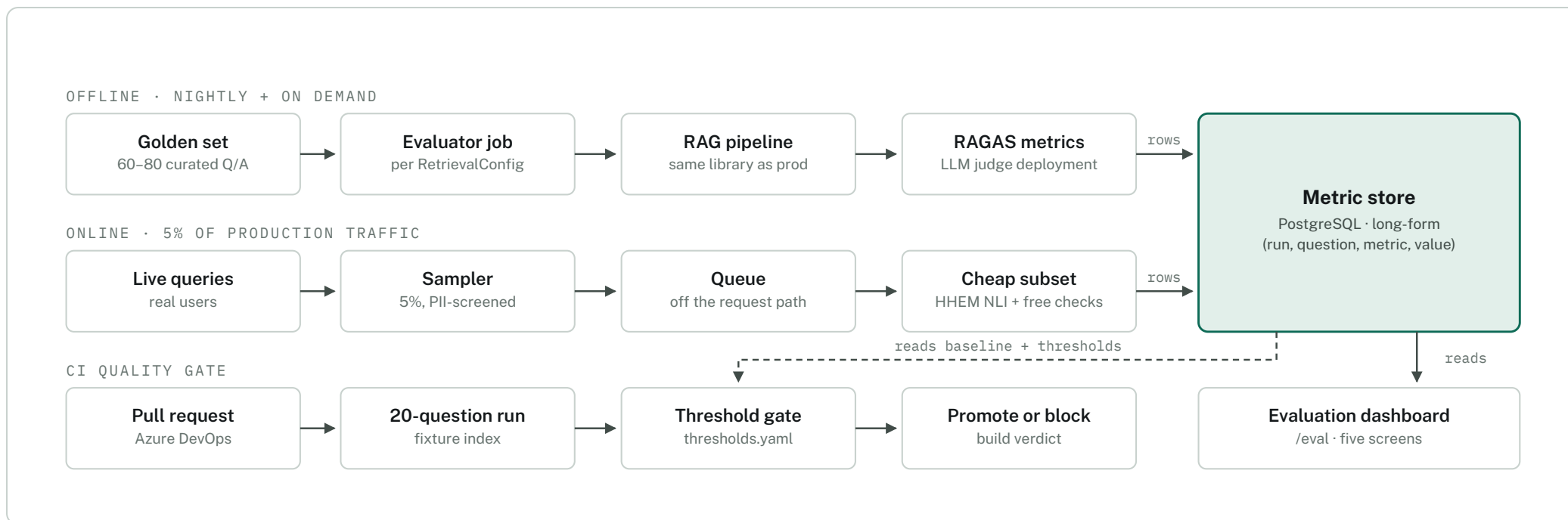


Figure 2. The offline path calls the same `rag_core` pipeline that serves production, so an eval score is a statement about shipped code. The online path deliberately runs a *cheaper* metric subset — an NLI cross-encoder for faithfulness plus the free deterministic checks — because paying an LLM judge on 5% of live traffic is how eval budgets get out of hand. The CI gate reads rather than writes: it compares this build's 20-question run against the stored baseline.

Building the golden set without spending a week on it

Bootstrap with RAGAS's `TestsetGenerator`, which builds a knowledge graph over the parsed documents and synthesises single-hop and multi-hop queries with references. Generate ~150 candidates, then curate down to 60–80 by hand — fixing wrong references, deleting the trivial, and recording gold chunk ids. Keep a `provenance` column marking synthetic vs. human-authored; synthetic questions skew easy and you want to be able to report the split.

Then add by hand the two categories generators are bad at, because they are the two that expose real failures:

- **Table-dependent questions** — "which segment had the largest operating income in FY23". Requires the table to have survived parsing, chunking, and the ranker's truncation window intact.
- **Unanswerable questions** — both the absurd ("the CEO's home address") and the plausible-but-absent ("FY26 guidance" in an FY23 filing). The second is far harder and is where hallucination actually happens.

Target mix: 30% single-fact, 25% multi-hop, 25% table-dependent, 20% unanswerable.

08 • METRICS

The RAGAS suite, and what each metric actually catches

Run the full suite nightly; run the cheap deterministic subset in CI and online.

Every metric below is surfaced in the UI — the tables here are the metric registry the dashboard renders from.

RETRIEVAL QUALITY – IS THE RIGHT CONTEXT REACHING THE MODEL

METRIC	NEEDS	COST	WHAT IT CATCHES HERE
LLMContextPrecisionWithReference	question, contexts, reference	LLM	Rank-weighted share of retrieved chunks that are relevant. Falls when <code>top_k</code> is too generous and filler pushes the real answer past the semantic ranker's read window.
LLMContextPrecisionWithoutReference	question, contexts, response	LLM	Same idea using the generated answer instead of a gold one — the version you can run on production traffic.
NonLLMContextPrecisionWithReference	contexts, reference_contexts	free	String-distance version. Deterministic and instant, so it belongs in the CI gate.
LLMContextRecall	question, contexts, reference	LLM	Whether every claim in the gold answer is supported by something retrieved. The metric that exposes a chunker splitting an answer across a boundary.
NonLLMContextRecall	contexts, reference_contexts	free	Gold chunk ids vs. retrieved ids — effectively hit-rate@k. Cheap regression tripwire.
ContextEntityRecall	contexts, reference	LLM	Entity coverage. Unusually informative on this corpus: tickers, fiscal periods and segment names are exactly the entities a vector-only retriever drops.
NoiseSensitivity	question, contexts, response, reference	LLM	How often the answer absorbs claims from <i>irrelevant</i> retrieved chunks. This is the specific risk hybrid retrieval creates by widening the candidate set — measure it before and after adding RRF.

GENERATION QUALITY – IS THE ANSWER SUPPORTED AND USEFUL

METRIC	NEEDS	COST	WHAT IT CATCHES HERE
Faithfulness	question, contexts, response	LLM	Decomposes the answer into claims and verifies each against the context. Your headline number and the CI gate's primary threshold.
FaithfulnesswithHHEM	contexts, response	NLI model	Same judgement from a hosted NLI cross-encoder instead of an LLM. Roughly two orders of magnitude cheaper — this is what the online 5% sample runs.
ResponseRelevancy	question, response	LLM + embed	Reverse-generates questions from the answer and compares to the original. Catches padded, evasive, or partially-answered responses that faithfulness happily passes.
ResponseGroundedness	contexts, response	LLM	Direct groundedness score; overlaps faithfulness but is cheaper and single-pass.

CORRECTNESS AGAINST A REFERENCE ANSWER

METRIC	NEEDS	COST	WHAT IT CATCHES HERE
FactualCorrectness	response, reference	LLM	Claim-level precision / recall / F1 against the gold answer. The best single "is it right" number; report the mode (precision vs. F1) you used.
SemanticSimilarity	response, reference	embed	Embedding cosine to the gold answer. Cheap, but generous — never quote it alone.
AnswerAccuracy / AnswerCorrectness	response, reference	LLM + embed	Blends factual and semantic agreement into one score for trend-watching.
ExactMatch, StringPresence	response, reference	free	Underrated on this corpus: assert the gold figure string ("-\$211.9 billion") is present. Zero cost, no judge variance.
BleuScore, RougeScore, NonLLMStringSimilarity	response, reference	free	Lexical baselines. Worth logging so you can show that LLM judges are telling you something the cheap metrics don't.

CUSTOM EVALUATORS – WHERE THE DOMAIN SHOWS UP

These are project-specific and are the ones that will distinguish the work. The first four are deterministic and free.

METRIC	DEFINITION	WHY IT MATTERS
citation_validity	Share of returned citations whose <code>chunk_id</code> was in context <i>and</i> whose <code>quoted_span</code> occurs verbatim in that chunk.	Programmatic hallucination detection. Gate at 0.95+.
citation_precision / recall	Cited chunks vs. the question's gold chunk ids.	Distinguishes "cited something real" from "cited the right thing".
numeric_exactness	Parse figures from the answer, normalise scale and unit (\$M vs \$B, basis points vs %), compare to gold within tolerance.	The failure mode that matters most in finance and that no generic metric measures. A 1000× scale error scores well on semantic similarity.
fiscal_period_correctness	Does the period the answer describes match the period asked about?	The exact error vector-only retrieval produces — right company, wrong year.
refusal_accuracy	On the unanswerable subset: correct refusals. Reported alongside false-refusal rate on the answerable subset.	Both directions. A system that refuses everything scores 1.0 on one and is useless.
table_faithfulness	Faithfulness restricted to questions whose gold chunk has <code>contains_table = true</code> .	A segment cut, not a new metric — and typically the ugliest number on the board. Surfacing it is the point.
AspectCritic / RubricsScore	RAGAS general-purpose judges with your own rubric.	Powers the manager-letter tone score against a 1–5 anchored rubric rather than bare positive/negative.

AZURE AI EVALUATION SDK – THE COMPLEMENTARY SET

Run alongside RAGAS, not instead of it. Its value is the service-backed evaluators and the fact that results land in Microsoft Foundry, which is worth a screenshot.

EVALUATOR	ADDS OVER RAGAS
GroundednessProEvaluator	Service-hosted groundedness with a reasoning string; a second opinion from a judge you didn't prompt yourself, which is useful when your own judge looks suspiciously generous.
RetrievalEvaluator, DocumentRetrievalEvaluator	NDCG, XDCG, fidelity and "holes" against graded relevance judgements — proper IR metrics rather than binary relevance.
Relevance / Coherence / Fluency / Similarity	Presentation-quality dimensions RAGAS does not cover.
ResponseCompleteness	Partial-answer detection on multi-hop questions.
IndirectAttackEvaluator (XPiA)	Genuinely relevant here. You ingest third-party PDFs; indirect prompt injection through document content is a real threat model for this system. Almost nobody tests it.
ContentSafety, ProtectedMaterial	Baseline safety coverage for the deployed endpoint.

OPERATIONAL METRICS – LOGGED PER REQUEST, CHARTED PER DAY

- **Latency, split by stage:** `retrieval_ms`, `rerank_ms`, `generation_ms`, `validation_ms` at p50/p95/p99. An aggregate number hides that the semantic ranker is your tail.
- **Spend:** prompt and completion tokens, embedding tokens, estimated cost per query, and *judge* cost per eval run — track that separately or it will surprise you.
- **Behaviour:** refusal rate, citation failure rate, chunks retrieved, filter-extraction hit rate, cache hit rate.
- **Freshness:** index lag, documents failing parse, chunks pending re-embed.

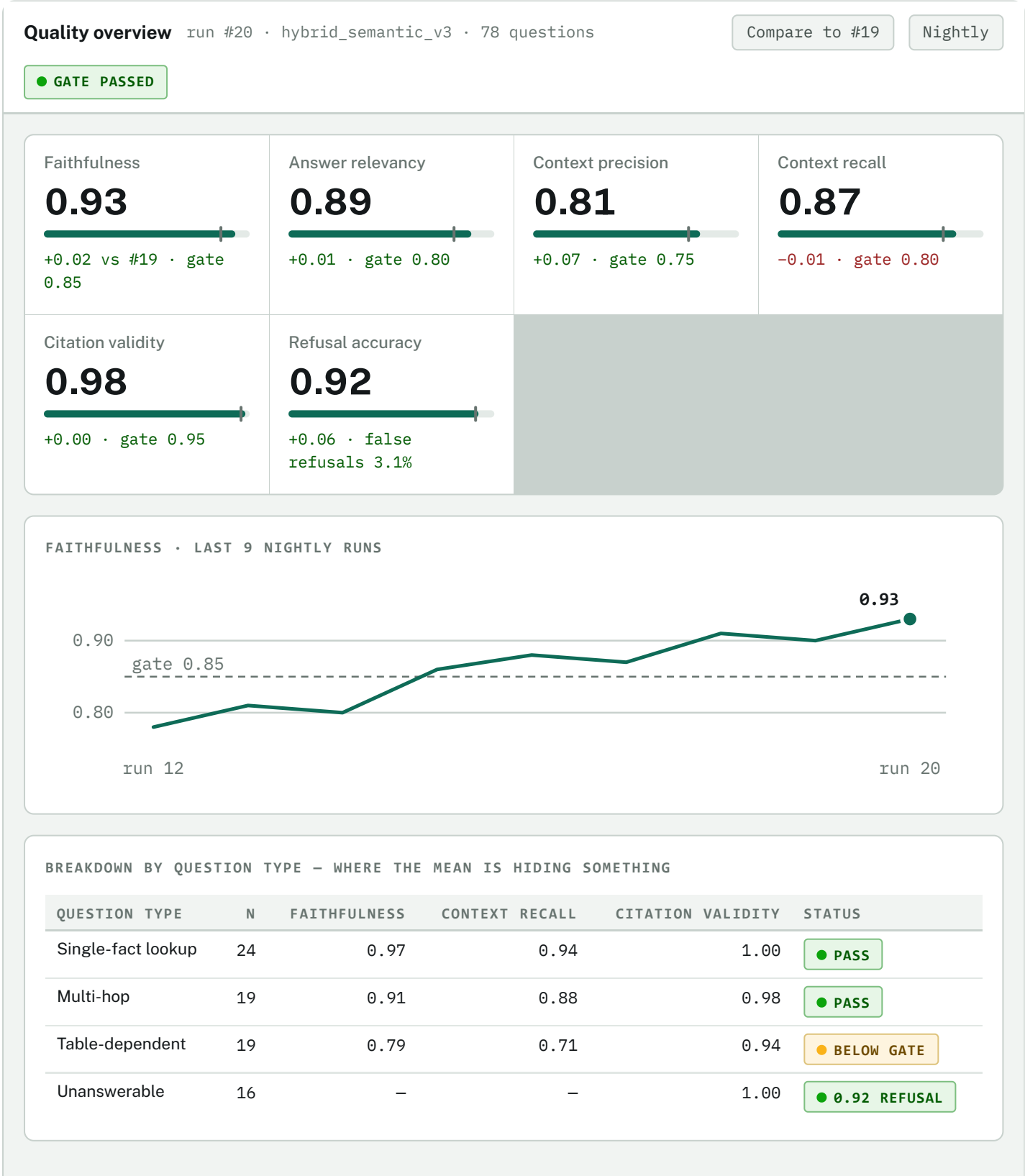
BUDGET THE JUDGE, SERIOUSLY

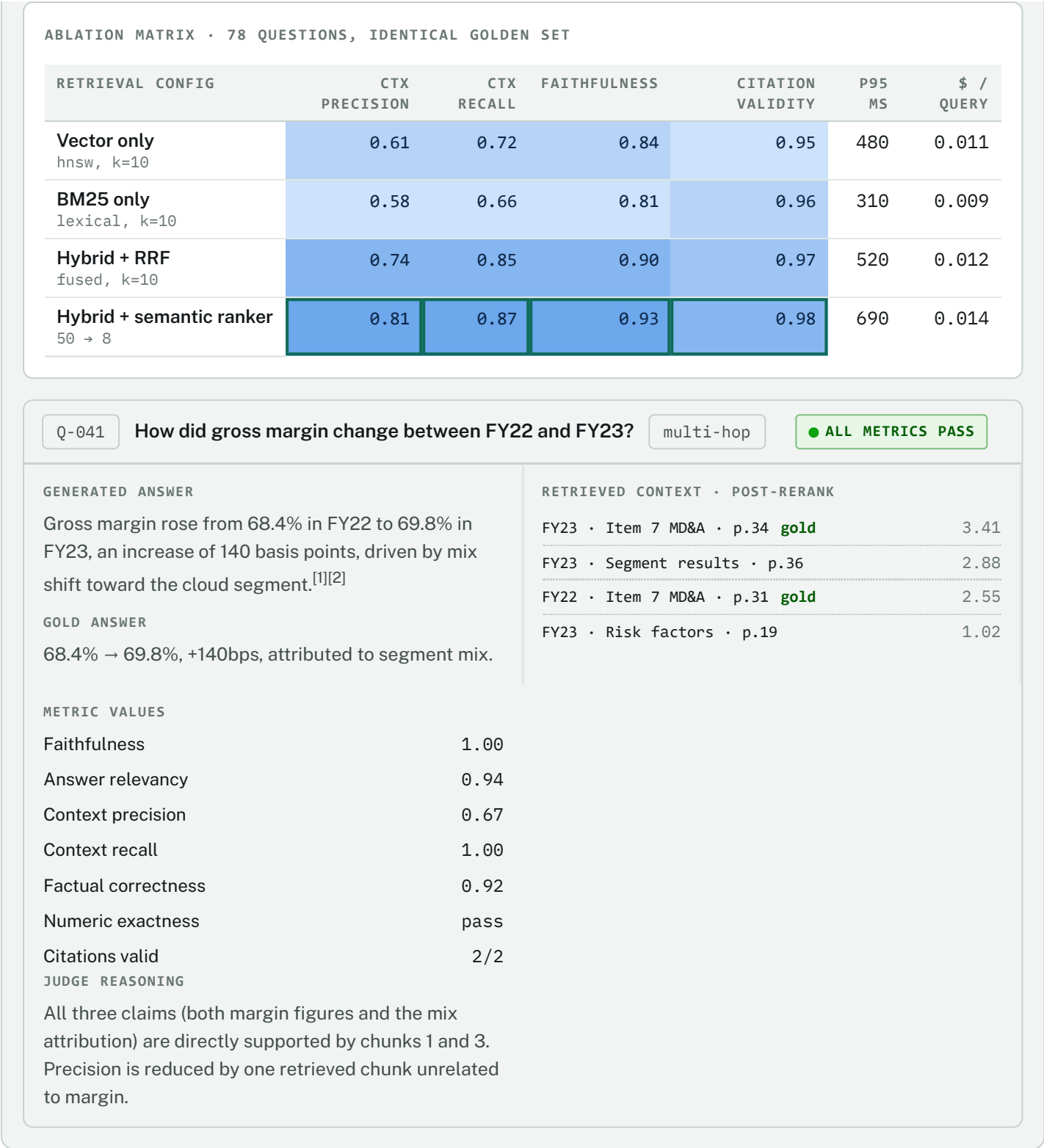
Faithfulness alone makes several judge calls per question (one to decompose claims, then verification). A full suite over 80 questions across 4 ablation configs is on the order of **5,000–8,000 judge calls per sweep**. Three mitigations, all cheap to build: cache verdicts keyed on `hash(question, answer, contexts, metric, judge_model)` so re-runs cost nothing; use the free deterministic metrics for CI; and reserve the LLM suite for nightly and for deliberate sweeps.

Putting the metrics on screen

Five screens. The design rule throughout: never show a mean without the segment breakdown underneath it, and never show a score without the threshold it is being judged against.

Below is the **Quality overview** at /eval , the screen that answers "is it good, and is it getting better".





The other four screens

- **Run detail** — every metric with its distribution, not just its mean. A metric averaging 0.85 because half the questions score 1.0 and half score 0.7 is a different system from one where everything scores 0.85, and only the histogram tells you which you have.
- **Compare** — two runs side by side with per-question deltas sorted by regression. After a prompt change this is the screen you actually use: it lists the six questions that got worse.
- **Question drilldown** — the panel shown above, full page, with the raw prompt and the per-retriever rank history for each chunk.
- **Production quality** — rolling window over sampled live traffic: faithfulness (NLI), citation failure rate, refusal rate, cost per query, and the top queries by low score. This is where you find out that real users ask things your golden set never imagined.

10 • PRODUCTION

Security, observability, delivery

Security

- **No keys in application code or config.** `DefaultAzureCredential` everywhere; the Container App's managed identity holds *Search Index Data Reader*, *Cognitive Services OpenAI User*, and *Storage Blob Data Reader*. Admin keys are used exactly once, by the deployment pipeline, from Key Vault.
- **Users authenticate with Entra ID;** the API validates the token and applies document-level filters from the user's claims, so per-user corpus scoping is enforced in the `$filter`, not in the prompt.
- **Treat ingested documents as untrusted input.** They are third-party PDFs entering an LLM context window. Run the indirect-attack evaluator over a sample, and never let retrieved content reach a system-role message.
- Private Endpoints on Search, OpenAI and Postgres if you want the full enterprise posture.

Observability

OpenTelemetry spans per stage, exported to Application Insights, with custom dimensions on every request: `config_id`, `retrieval_latency_ms`, `rerank_latency_ms`, `generation_latency_ms`, `chunks_retrieved`, `prompt_tokens`, `completion_tokens`, `estimated_cost`, `citation_validation_failures`, `refusal`, `filter_extracted`, `model_deployment`.

Then four KQL queries worth building and screenshotting: p95 latency by stage over time; token spend per day by deployment; citation-failure rate as a 7-day trend; and refusal rate split by whether a metadata filter was applied — that last one usually reveals that over-aggressive filtering, not the model, is causing refusals.

CI/CD with a quality gate

Pipeline stages: lint → unit tests (chunker boundaries, table reconciliation, citation validator, metadata extraction, numeric normalisation) → integration tests against a small fixture index → build containers → **eval gate** → deploy.

The gate runs 20 questions against the fixture index and fails the build if any threshold in `thresholds.yaml` is breached: faithfulness < 0.85, citation validity < 0.95, context recall more than 3 points below the stored baseline, or p95 latency above budget. Use only free and NLI-based metrics here so the gate costs cents and runs in under two minutes.

A nightly pipeline runs the full LLM suite over the complete golden set across all configs and writes a new baseline. Wire a failure to a notification — a regression you don't hear about is a regression you ship.

Nine phases, each with an exit test

The order is chosen so that nothing is tuned before it can be measured — but also so the eval harness is not built before there is anything to evaluate. Phases 5 and 6 are the ones people skip; they are the ones worth protecting.

0

3 DAYS

Foundation

Monorepo, uv workspace, Bicep modules for resource group, Storage, AI Search, Azure OpenAI, PostgreSQL, Key Vault and the Container Apps environment. Entra app registration. CI skeleton.

EXIT TEST

One command stands up an empty but complete environment, and the API's health check performs a managed-identity-authenticated call to Search.

1

WEEK 1

Ingestion & parsing

Blob layout, Event Grid → queue → job, Document Intelligence `prebuilt-layout` with async polling and resume, cross-page table reconciliation, the `openpyxl` path for spreadsheets, parsed cache.

EXIT TEST

200 documents parsed; unit tests merge ten known page-spanning tables correctly; re-running the job over the same corpus is a no-op.

2

WEEK 1-2

Chunking & indexing

Layout-aware chunker with the never-split-a-table rule and context headers, full metadata set, index schema deployed as code, batched embedder with exponential backoff, upsert path.

EXIT TEST

Chunk size is a `RetrievalConfig` parameter, and a full re-embed from the parsed cache completes without re-parsing a single document.

3

WEEK 2-3

Retrieval, generation, citations

Single-request hybrid query with RRF, filter extraction, semantic ranker, citation-constrained prompt with structured output, the programmatic validator, trace persistence.

EXIT TEST

A test that injects a fabricated citation into a model response is caught and dropped by the validator; `/api/search` returns per-retriever scores for every candidate.

4

WEEK 3

Chat frontend

Next.js shell, streaming chat with typed SSE events, interactive citation markers, pdf.js document viewer with bounding-box overlay, corpus browser, per-answer quality footer.

EXIT TEST

Clicking any citation opens the correct document page with the cited region highlighted.

5

WEEK 4

Golden set & eval harness

RAGAS testset generation, curation to 60–80 questions with the four-type mix, RAGAS wired to a dedicated judge deployment, custom evaluators, judge-verdict cache, run persistence, Typer CLI.

EXIT TEST

A single command produces a complete stored run; two runs of different configs are comparable with one SQL query.

6

WEEK 4-5

Evaluation UI

The five screens: overview with thresholds and trend, run detail with distributions and segment breakdown, ablation matrix, comparison view sorted by regression, question drilldown.

EXIT TEST

Every number on screen comes from the API; no static fixtures remain in the frontend.

7

WEEK 5

Ablations & tuning

Four retrieval configs; chunk-size sweep at 300 / 500 / 800 / 1200; embedding dimensions 3072 vs 1024; optional local cross-encoder after the semantic ranker.

EXIT TEST

The results table exists and the negative results are kept in it — "the cross-encoder cost 180ms and gained 0.004" is a finding, not a failure.

8

WEEK 6

Production wrap

Key Vault, managed identity and RBAC data-plane roles; OpenTelemetry to App Insights; KQL dashboard; Azure DevOps CI/CD with the eval gate; online 5% sampling loop.

EXIT TEST

Deliberately open a PR that degrades the prompt, and show the pipeline blocking it on the faithfulness threshold. That screenshot is the single best artifact in the project.

9

WEEK 6-7

Finance capabilities

Document classification (few-shot LLM vs. fine-tuned DistilBERT, compared on accuracy *and* cost per document); structured extraction into a typed model validated by accounting identities; manager-letter summarisation with an anchored 1–5 tone rubric tracked per author over time.

EXIT TEST

Extraction flags identity violations rather than silently accepting them, and the classifier comparison table shows both accuracy and unit cost.

12 • COST & RISK

What it costs and what will bite

Running cost, order of magnitude

For a 200-document corpus at roughly 80 pages each. **Verify every figure against current Azure pricing for your region** before committing — these are planning estimates for sizing decisions, not quotes.

ITEM	SHAPE	ROUGH SCALE	NOTE
Document Intelligence	one-off, per page	~16,000 pages	The largest single ingestion cost. Reason enough for the parsed cache.
Embeddings	one-off, per token	~10M tokens	Trivially cheap next to parsing. Re-embedding is affordable; re-parsing is not.
AI Search	monthly, tier	Basic + semantic	Fixed floor. The semantic ranker is separately billed — enable deliberately.
Generation	per query	low	Dev traffic only; dominated by context size, so chunk size drives it.
LLM judge (eval)	per sweep	5,000–8,000 calls	The variable that surprises people. Cache verdicts; keep CI on free metrics.
PostgreSQL, Container Apps, Blob	monthly	small	Burstable tier and scale-to-zero.

Risks, in the order they will actually hurt

RISK	MITIGATION
Azure OpenAI TPM quota is per-region and per-deployment; bulk embedding hits it immediately.	Check allocation before sizing the ingest job. Batch, back off exponentially, and give the judge its own deployment so eval never starves serving.
Document Intelligence async polling fails mid-run and restarts the whole corpus.	Per-document state machine with a resume point; the parsed cache makes a restart cheap.
Tables spanning page breaks return as two tables.	Explicit reconciliation on column count and header match, with unit tests over known cases.
The semantic ranker reads only ~2,000 tokens of the content field and silently truncates.	Keep chunks in the 500–800 token band; assert an upper bound in the chunker's tests.
LLM judges are non-deterministic; a 0.02 movement may be noise.	Fix judge temperature and version, pin the model deployment, and run the gate on a threshold band rather than exact equality. Record judge model on every run.
Tuning against 78 questions overfits to them.	Hold out 20 questions never used for tuning; report on both. Add production queries found by online eval back into the set over time.
Eval cost grows quietly with every added metric and config.	Judge-verdict cache, deterministic metrics in CI, full suite nightly only.
Indirect prompt injection through ingested third-party documents.	Retrieved content never enters a system message; run the XPIA evaluator over a sample of the corpus.
Portal naming drift — the console is now branded Microsoft Foundry (renamed from Azure AI Foundry, Jan 2026).	Write both forms in the README and CV. Azure AI Search and Azure AI Document Intelligence kept their names.

IF YOU BUILD ONLY THREE THINGS

The ablation matrix with real numbers, the citation validator, and the CI pipeline blocking a build on a faithfulness threshold. Everything else in this plan is supporting structure for those three — they are the parts that demonstrate you can *operate* a RAG system rather than assemble one.